

Ten-Week Research Roadmap

Energy-Aware Expert Routing in Mixture-of-Experts Models

Prepared for Hrishabh Gupta

Prepared by Dr. Raj Dandekar

March 12, 2026

Abstract

This research introduces a novel energy-aware routing mechanism for Mixture-of-Experts (MoE) models, augmenting the standard top-k gating function with a real-time energy penalty term, $\lambda \cdot e$, derived from GPU power states. Our methodology trains the MoE model using a multi-objective loss that explicitly balances task performance with total energy consumption, representing a new approach in the Green AI space. The primary evaluation will be conducted on the C4 (Colossal Clean Crawled Corpus) dataset, measuring the trade-off between language modeling perplexity and energy consumption in Joules per inference. Key deliverables include a PyTorch implementation of the energy-aware router, a comprehensive analysis of the quality-energy Pareto frontier, and an investigation into how the router learns to allocate tokens of varying complexity. The target output is a high-impact workshop paper for a venue such as the Sustainable AI workshop at NeurIPS or SustaiNLP at EMNLP.

Tools for Your Research

Claude Code is an AI coding assistant that runs in your terminal. It can help you write experiment scripts, debug code, set up environments, process data, and generate visualizations — all through natural language.

Getting Started:

- Install: `npm install -g @anthropic-ai/claude-code`
- Use: `cd your-project && claude`
- When to use: stuck on code, setting up environments, debugging, writing scripts, data processing, creating plots

Contents

- 1 Scope & Research Questions
- 2 Primary Dataset / Methodology
- 3 Milestone 1 (Weeks 1–2): Literature Review & Foundations
- 4 Milestone 2 (Weeks 3–4): Data/Environment Setup & Baselines
- 5 Milestone 3 (Weeks 5–6): Core Experiments & Ablations
- 6 Milestone 4 (Weeks 7–8): Evaluation & Analysis
- 7 Milestone 5 (Weeks 9–10): Manuscript Writing & Submission
- A Appendix A: Literature Tracking Spreadsheet Columns
- B Appendix B: Metric Definitions
- C Appendix C: Project Folder Layout

1 Scope & Research Questions

Goal. This project aims to develop and evaluate a novel energy-aware routing mechanism for Mixture-of-Experts (MoE) models, directly integrating sustainability into the model architecture. The system will modify the standard top-k gating network to incorporate a real-time energy cost signal, e_{GPU} , for each expert's host GPU, controlled by a hyperparameter λ . Evaluation will focus on constructing and analyzing the Pareto frontier of task performance (perplexity on C4, average score on GLUE) versus energy consumption (Joules), demonstrating a favorable and tunable trade-off compared to energy-agnostic routing baselines.

Core questions.

1. RQ1: How does incorporating a real-time energy penalty $\lambda \cdot e_{\text{GPU}}$ into the MoE routing decision affect the trade-off between model quality (measured by perplexity on C4 and accuracy on GLUE) and total energy consumption during inference? We will measure this by systematically varying λ from $1e-5$ to $1e-1$ and plotting the resulting Pareto frontier, comparing it against a state-of-the-art baseline MoE (Switch Transformer) which represents the $\lambda=0$ case.
2. RQ2: Does an energy-aware router learn to send semantically or computationally 'simpler' tokens to low-power experts and 'harder' tokens to high-power experts? Our testable hypothesis is that as λ increases, tokens with lower initial routing uncertainty (measured by the entropy of the affinity scores before the energy penalty is applied) will be preferentially routed to GPUs simulated to be in a low-power Dynamic Voltage/Frequency Scaling (DVFS) state.
3. RQ3: What is the measurable performance degradation on a suite of downstream tasks (the GLUE benchmark) when targeting specific, aggressive energy reduction budgets (e.g., 20%, 30%, 40%) compared to an energy-agnostic MoE router? This will be quantified by fine-tuning our energy-aware MoE with different fixed λ values and reporting the average GLUE score for each corresponding energy budget, establishing a practical quality-cost curve.

2 Primary Dataset / Methodology

Primary dataset (use this specifically).

C4 (Colossal Clean Crawled Corpus) - en.noclean split (2020): The C4 dataset is a massive, cleaned version of Common Crawl containing approximately 750GB of English text, making it a standard for large-scale language model pre-training. Its scale and diversity are essential for robustly training MoE models and obtaining reliable perplexity measurements. The suitability comes from its use in foundational MoE papers like Switch Transformers, allowing for direct comparison. A known limitation is the presence of residual noise and potential biases inherent in web-scraped data, which requires careful data loading and filtering.

Optional validation sets (stress-tests, not mandatory):

- GLUE Benchmark (2019): A collection of nine sentence-level language understanding tasks used to evaluate the downstream generalization of our pre-trained models.
- WikiText-103 (2016): A high-quality, medium-sized dataset (~500MB) for initial, faster prototyping and debugging of the routing mechanism and energy monitoring scripts.
- The Pile (2021): An 825GB diverse, open-source language modeling data set that can be used to test the robustness of the routing mechanism on a different data distribution.

3 Milestone 1 (Weeks 1–2): Literature Review & Foundations

Objectives

- Read and synthesize 12-15 core papers on MoE, Green AI, and systems for ML, taking detailed, structured notes.
- Maintain an Excel or Notion tracking sheet with 5-7 sentence summaries for over 15 additional relevant papers to build a comprehensive knowledge base.
- Identify and document 3-4 specific baseline approaches, including standard MoE implementations and theoretical lower-bounds, to compare against our proposed method.

Core Reading List

1. Shazeer, N. et al. (2017). Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. ICLR.
2. Fedus, W. et al. (2022). Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. JMLR.
3. Lepikhin, D. et al. (2020). GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. ICLR.
4. Riquelme, C. et al. (2021). Beyond Uniform Scaling: Exploring Expert Diversity in MoEs. arXiv.
5. Clark, A. et al. (2022). What Do Experts Do in Sparsely-Gated Mixture-of-Experts? An Analysis of Expert Specialization. EMNLP.
6. Puigcerver, J. et al. (2023). The unreasonable effectiveness of a single expert in a Mixture-of-Experts. NeurIPS.
7. Schwartz, R. et al. (2020). Green AI. Communications of the ACM.
8. Patterson, D. et al. (2021). Carbon Emissions and Large Neural Network Training. arXiv.
9. Lannelongue, L. et al. (2021). Green Algorithms: Quantifying the Carbon Footprint of Computation. Advanced Science.
10. Henderson, P. et al. (2020). Towards the Systematic Reporting of the Energy and Carbon Footprints of Machine Learning. JMLR.
11. De Vogeleer, K. et al. (2022). PyJoules: A Python Library for Energy Consumption Monitoring. Journal of Open Source Software.
12. NVIDIA. (2023). NVIDIA System Management Interface (nvidia-smi) Documentation.
13. Hooker, E. (2021). The Hardware Lottery. Communications of the ACM.
14. Mustafa, B. et al. (2022). Multimodal-MoE: An effective and efficient framework for multimodal representation learning. NeurIPS.
15. Kool, W. et al. (2024). MoE-Mamba: Efficient Selective State Space Models with Mixture of Experts. ICLR.

Concrete Steps / Tasks

1. Set up a Zotero or Mendeley library, tagging each paper with keywords like 'MoE-routing', 'Green-AI', 'energy-measurement', and the relevant RQs.

2. 2. Read the three foundational MoE papers (Shazeer 2017, Lepikhin 2020, Fedus 2022) to deeply understand the architecture, training challenges (e.g., load balancing), and implementation details.
3. 3. Read five recent papers (2022-2024) on MoE analysis and Green AI to identify the current state-of-the-art and research gaps.
4. 4. Create a detailed comparison table in Excel comparing different MoE routing strategies (e.g., Top-k, hashing), their load balancing mechanisms, and reported performance.
5. 5. Write a 2-3 page literature review memo that synthesizes the key challenges in MoE routing and the methodologies for measuring energy in ML, explicitly stating the novel contribution of this project.
6. 6. Identify and document the primary gap: No existing work incorporates real-time, heterogeneous energy costs into the MoE routing decision function.

Deliverables

- literature_review/review_memo.pdf (2-3 pages summarizing findings and positioning the project)
- literature_review/papers.xlsx (with 15+ detailed summaries, including methodology and critiques)
- literature_review/comparison_table.xlsx (comparing MoE architectures and Green AI methods)

Acceptance Check

Literature review memo clearly articulates the research gap and project contribution, supported by the detailed paper summaries and comparison table.

Risks & Mitigations

Risk: Overwhelmed by the volume of literature.

Mitigation: Focus first on papers cited by the foundational works (e.g., Switch Transformers). Use a 'triage' reading approach: abstract/conclusion first, then methods, then full read if highly relevant.

Risk: Missing very recent, un-indexed papers.

Mitigation: Set up Google Scholar and Semantic Scholar alerts for keywords like 'Mixture of Experts', 'Conditional Computation', and 'Sustainable AI'. Follow key researchers on social media.

4 Milestone 2 (Weeks 3–4): Data/Environment Setup & Baselines

Objectives

- Establish a fully reproducible development environment with all required libraries, including specialized ones for MoE and GPU monitoring.
- Download, preprocess, and set up a data loading pipeline for the C4 dataset, including a 1% sampled version for rapid development.
- Implement and benchmark two critical baseline models: a standard energy-agnostic MoE and a random-routing lower bound.

Concrete Steps / Tasks

1. Create a conda environment and a `requirements.txt` file with specific versions of `torch`, `transformers`, `deepspeed`, `datasets`, and `pynvml` for NVIDIA GPU monitoring.
2. Download the C4 `en.noclean` split using the `datasets` library. Write and validate a script to create a 1% random subsample for development and save it to disk.
3. Implement `src/data_loader.py` with efficient tokenization and batching, capable of streaming from the full dataset to avoid memory issues.
4. Implement Baseline 1: A standard Switch Transformer using the DeepSpeed-MoE library. Configure it as a decoder-only model for language modeling. This will serve as our primary ($\lambda=0$) baseline.
5. Implement Baseline 2: A 'Random Router' where tokens are uniformly randomly assigned to experts. This establishes a performance lower-bound and helps debug the MoE container.
6. Develop a utility script `src/utils/energy_monitor.py` that uses the `pynvml` library to periodically query `nvidia-smi` for the power draw of each GPU, integrating this data into a logging framework like `wandb` to calculate total energy in Joules for each run.

Deliverables

- `environment.yml` or `requirements.txt` file for reproducibility.
- `data/README.md` describing the C4 dataset, preprocessing steps, and sampling strategy.
- `src/baselines/switch_transformer.py` and `src/baselines/random_router.py` with implementation code.
- `results/baselines.csv` containing perplexity scores and measured energy consumption for both baselines on the 1% C4 subset.

Tip — Use Claude Code: If setting up the environment feels complex, use Claude Code to help install dependencies, configure your project structure, and write boilerplate code.

Acceptance Check

Both baseline models run end-to-end on the 1% C4 data subset. The Switch Transformer baseline achieves a perplexity score within 10% of reported benchmarks, and the energy monitor logs plausible energy values.

Risks & Mitigations

Risk: The complexity of setting up distributed MoE frameworks like DeepSpeed.

Mitigation: Start with the simplest official DeepSpeed-MoE tutorial and incrementally modify it. Budget at least 3 full days for environment setup and debugging.

Risk: Inaccurate or noisy energy measurements from `nvidia-smi`.

Mitigation: Average power readings over a 1-second window to smooth out instantaneous spikes. Validate by running a high-intensity workload (e.g., `torch.matmul`) and ensuring the power draw matches the GPU's TDP.

5 Milestone 3 (Weeks 5–6): Core Experiments & Ablations

Objectives

- Implement the core novel contribution: the energy-aware MoE routing layer and the associated multi-objective loss function.
- Conduct a systematic grid of ablation studies to understand the sensitivity of the model to key hyperparameters, especially the energy penalty weight λ .
- Thoroughly document all experimental runs and initial findings to identify the most promising configurations for final evaluation.

Concrete Steps / Tasks

1. Implement the `EnergyAwareGating` module in `src/models/energy_aware_moe.py`, inheriting from a standard gating class and adding the energy penalty logic.
2. Write an experiment runner script (`run_experiment.py`) that accepts configuration files (e.g., YAML) to easily launch ablation studies.
3. Integrate the energy monitor to pass the real-time energy vector to the model during the forward pass. For simulation, this can be a pre-defined vector.
4. Run the full ablation grid (A1-A5) on the 1% C4 subset to quickly explore the parameter space.
5. Analyze the `wandb` logs, focusing on the Perplexity vs. Joules scatter plot for different λ values to identify the emerging Pareto frontier.

Exact Ablations (one factor at a time)

ID	Factor	Levels
A1	Energy Penalty (λ)	1e-5, 5e-5, 1e-4, 5e-4, 1e-3, 1e-2, 1e-1 (log scale)
A2	Energy State Simulation	'Static' (assign fixed low/high power states to GPUs), 'Dynamic' (randomly change GPU power states every 100 steps)
A3	Number of Experts	8, 16, 32 (to test scalability of the approach)
A4	Top-k Routing	k=1, k=2 (to see if routing to multiple experts provides useful flexibility)
A5	Load Balancing Loss Weight (α)	0.0, 0.01, 0.05 (to study interaction with the energy term)

Implementation Notes

- Use random seeds {42, 123, 456} for all final experiment runs to ensure statistical robustness.
- The core modification is in the gating network's forward pass: `logits = (X @ W_g) - self.lambda * energy_costs`. The `energy_costs` vector `[e1, e2, ..., eN]` must be fetched from the energy monitor and broadcasted to the batch dimension.
- Log all metrics, hyperparameters, and expert utilization histograms to Weights & Biases (wandb) for real-time monitoring and post-hoc analysis.

- The energy cost vector should be treated as a non-differentiable constant in the forward pass to avoid backpropagating through the energy measurement process.

Deliverables

- `src/models/energy_aware_moe.py` containing the core model implementation.
- `configs/` folder with YAML files for each ablation experiment.
- `results/ablations/` directory containing raw CSV logs for all runs.
- `experiments.md`: A detailed markdown document with initial plots and analysis of the ablation results, identifying the best configurations.

Tip — Use Claude Code: Use Claude Code to help write experiment scripts, debug issues, and process results into structured CSVs.

Acceptance Check

A clear, well-defined Pareto frontier is observable in the plot of perplexity vs. energy as λ is varied, demonstrating the method's core trade-off mechanism is working.

Risks & Mitigations

Risk: GPU Out-of-Memory (OOM) errors with larger numbers of experts.

Mitigation: Use DeepSpeed's ZeRO-Offload to offload parameters/optimizer states to CPU memory. Reduce batch size as a last resort.

Risk: The energy term completely overwhelms the affinity score, leading to model collapse.

Mitigation: Start with very small values of λ (e.g., $1e-6$) and increase gradually. Implement gradient clipping to ensure training stability.

6 Milestone 4 (Weeks 7–8): Evaluation & Analysis

Objectives

- Run the final, best-performing model configurations on the full validation dataset to obtain publication-ready results.
- Compute all primary and secondary metrics with confidence intervals to ensure statistical significance.
- Generate all figures and tables required for the manuscript, including a deep qualitative analysis of the model's behavior.

Concrete Steps / Tasks

1. 1. Select the 3-5 most promising hyperparameter configurations (especially different λ values) from the ablation phase.
2. 2. Run these final configurations on the full C4 validation set with 3 different random seeds to compute mean and standard deviation for perplexity and energy.
3. 3. Fine-tune the pre-trained checkpoints (baseline and 3 best models) on the GLUE benchmark and compute the average score.
4. 4. Generate all figures using `matplotlib`/`seaborn` and save them as high-resolution PDFs.
5. 5. Create LaTeX-formatted tables for the main numerical results using a tool like `booktabs` for professional quality.
6. 6. Conduct the error analysis for RQ2: extract tokens routed to the lowest- and highest-power experts and analyze their linguistic properties (e.g., word frequency via TF-IDF, sentence complexity via a parser).

Metrics

Perplexity (PPL): Calculated as $\exp(\text{cross-entropy loss})$ on the C4 validation set. The primary metric for language modeling quality.

GLUE Score: The macro-average of scores across the 9 GLUE benchmark tasks after fine-tuning.

Inference Energy (Joules): Total energy consumed to process a fixed-size evaluation set, calculated by integrating power draw over time.

Expert Utilization Gini Coefficient: A measure of routing inequality. A score of 0 indicates perfect load balance; 1 indicates all tokens go to one expert.

Energy-Quality Pareto Frontier: A 2D plot of (Joules, Perplexity) pairs for different λ values, illustrating the optimal trade-offs.

Visualizations

- Figure 1: The main results plot showing the Perplexity-vs-Energy Pareto frontier. It must compare our method across 5-6 λ values against the energy-agnostic baseline.
- Figure 2: A bar chart showing average GLUE score vs. percentage of energy saved for the baseline and 3 of our best models.

- Figure 3: A qualitative analysis table showing examples of 'simple' sentences (e.g., repetitive, common phrases) routed to low-power experts and 'complex' sentences (e.g., rare words, complex syntax) routed to high-power experts.
- Figure 4: A heatmap of expert utilization for the baseline vs. a high- λ model, where experts are sorted by their assigned power state, to visually confirm the routing shift.
- Figure 5: A line plot showing the sensitivity of perplexity to the number of experts for both the baseline and an energy-aware model.

Deliverables

- results/final_results.csv containing all final metrics with confidence intervals.
- figures/*.pdf directory with all publication-quality visualizations.
- tables/*.tex directory with all LaTeX-ready results tables.
- analysis/qualitative_analysis.md detailing the investigation into routing decisions with concrete examples.

Tip — Use Claude Code: Use Claude Code to help create visualization scripts, format results tables, and generate publication-quality figures.

Acceptance Check

All figures and tables are complete and clearly support the main claims of the project. The Pareto frontier in Figure 1 shows a statistically significant improvement over the baseline.

Risks & Mitigations

Risk: Results are not statistically significant or the improvement is marginal.

Mitigation: Report effect sizes in addition to p-values. Pivot the paper's focus to be more of an analysis of the challenges of energy-aware routing, even if the gains are small.

Risk: Fine-tuning on GLUE is computationally expensive and time-consuming.

Mitigation: Start fine-tuning runs early in the week. Consider evaluating on a smaller subset of GLUE tasks (e.g., SST-2, MRPC, QQP) if time is critical.

7 Milestone 5 (Weeks 9–10): Manuscript Writing & Submission

Objectives

- Write a complete, compelling 4-6 page manuscript following the guidelines of a top-tier workshop (e.g., NeurIPS format).
- Prepare a clean, documented, and reproducible code release for submission and potential open-sourcing.
- Submit the manuscript and supplementary materials to the target venue before the deadline.

Concrete Steps / Tasks

1. Create an Overleaf project using the official NeurIPS LaTeX template.
2. Draft the Introduction and Methods sections first, as they are based on well-defined work.
3. Integrate the figures and tables from M4 into the Results section and write the surrounding narrative, focusing on the story told by the Pareto frontier.
4. Write the Abstract, Related Work, and Conclusion sections.
5. Clean up the project's GitHub repository: add a detailed README.md with setup and reproduction instructions, license the code (e.g., MIT), and archive the final model checkpoints.
6. Share the draft with the mentor and peers for a full review cycle. Incorporate feedback.
7. Perform a final round of proofreading and submit the paper and supplementary materials (code) to the target workshop.

Deliverables

- manuscript/main.pdf (final, camera-ready version of the paper).
- manuscript/supplementary.zip containing the code, README, and any extra figures or details.
- A public GitHub repository with the cleaned, documented code and instructions.

Tip — Use Claude Code: Use Claude Code to help format tables, generate LaTeX, organize supplementary materials, and proofread your manuscript.

Acceptance Check

Manuscript is submitted to the target venue by the deadline, and the accompanying code repository is complete and allows for straightforward reproduction of the key results.

Risks & Mitigations

Risk: The story is unclear or the contributions are not well-articulated.

Mitigation: Create a one-paragraph 'story' of the paper before writing and ensure every section supports it. Get feedback from people outside the immediate project area to check for clarity.

Risk: Running out of time before the submission deadline.

Mitigation: Set a personal deadline 3 days before the official one. Write sections as they are completed (e.g., write the Methods section during M3), don't wait until the final two weeks.

Timeline Overview

Milestone	Weeks	Key Deliverables
M1: Literature Review	1–2	review_memo.pdf, papers.xlsx, comparison_table.xlsx
M2: Setup & Baselines	3–4	environment.yml, results/baselines.csv, src/utils/energy_monitor.py
M3: Experiments	5–6	src/models/energy_aware_moe.py, results/ablations/, experiments.md
M4: Evaluation	7–8	results/final_results.csv, figures/*.pdf, tables/*.tex
M5: Manuscript	9–10	manuscript/main.pdf, Public GitHub repository, README.md

Appendix A: Literature Tracking Spreadsheet Columns

- Paper Title
- Authors
- Year
- Venue
- Problem/Task
- Dataset(s)
- Method Summary (5-7 sentences)
- Key Metrics
- Key Results
- Limitations/Critique
- Code Available?
- Relevance (1-5)
- Notes/Ideas

Appendix B: Metric Definitions

Perplexity. $\text{PPL}(X) = \exp(- (1/N) * \sum \log P(x_i | x_1 \dots x_{i-1}); \theta)$

Joules. $E = \int P(t) dt \approx \sum P_i * \Delta t$, where P_i is the power in Watts at interval i and Δt is the interval duration in seconds.

Gini Coefficient. $G = (\sum \sum |c_i - c_j|) / (2 * n^2 * \mu)$, where c_i is the number of tokens routed to expert i , n is the number of experts, and μ is the mean of c .

GLUE Score. Macro-average of the primary metrics for CoLA (MCC), SST-2 (Acc), MRPC (F1), STS-B (Spearman), QQP (F1), MNLI (Acc), QNLI (Acc), RTE (Acc).

Appendix C: Project Folder Layout

energy-aware-moe/ ■■■ data/ ■ ■■■ raw/ ■ ■■■ processed/ ■ ■■■ README.md ■■■ src/ ■ ■■■
models/ ■ ■ ■■■ energy_aware_moe.py ■ ■■■ baselines/ ■ ■ ■■■ switch_transformer.py ■ ■■■ utils/ ■
■ ■■■ energy_monitor.py ■ ■■■ data_loader.py ■ ■■■ run_experiment.py ■■■ configs/ ■ ■■■
baseline.yaml ■ ■■■ ablations/ ■■■ notebooks/ ■ ■■■ analysis.ipynb ■■■ results/ ■ ■■■ ablations/ ■
■■■ final/ ■ ■■■ baselines.csv ■■■ figures/ ■■■ tables/ ■■■ manuscript/ ■■■ literature_review/ ■■■
environment.yml ■■■ requirements.txt ■■■ README.md